

Flipping the Script: Out-of-Order Transmission with In-Order Delivery for Maximized Link Utilization

Max Franke*, Vamsi Addanki[†] and Stefan Schmid*

*TU Berlin

[†]Purdue University

Abstract—Oblivious reconfigurable networks (ORNs) constantly change their topology, offering high throughput and path diversity over time. However, existing approaches, including Valiant Load Balancing, UCMP, and RotorLB, risk out-of-order packet delivery, limiting compatibility with transports such as RoCEv2. We propose to flip the script: instead of repairing reordering at the receiver, we control transmission order at the source. Later chunks are sent first over slower paths and earlier chunks later over faster paths, so that assignments satisfying a protection-interval condition arrive in order. We call this approach MPOA (Multiple Paths, Ordered Arrival). MPOA uses source-ToR buffering and schedule-derived path rankings to exploit multiple temporal paths while preserving in-order arrival under bounded latency-estimation error. Simulations show higher average link utilization than UCMP, making MPOA a promising primitive for demand-oblivious datacenter fabrics.

I. INTRODUCTION

Traditionally, datacenter networks are built out of electrical packet switches, arranged in a hierarchical topology [1]. However, the explosive growth in datacenter network traffic and the slowdown in Moore’s law bandwidth scaling for electrical interconnects, prompted significant research efforts on integrating optical switching into datacenters [2], [3], [4], [5], [6], [7], [8], [9]. A variety of network designs have been proposed in the recent past, with optical circuit reconfigurations ranging from more than an hour timescale [5], all the way to nanosecond timescale switching [2]. Leveraging fast reconfigurable optical switches, the design paradigm of oblivious circuit switching gained traction in the recent years across the research community [3], [4], [2], [10], [11], [12] — particularly for the simplicity of the switching schedule without explicitly requiring a control plane. Yet, the integration of such a dramatically new switching fabric into existing datacenters is not without challenges.

First, in oblivious reconfigurable networks (ORNs), multiple paths are available between any pair of hosts, with different paths often having different latencies. This contrasts with traditional datacenter networks, where the topology is fixed and multiple paths typically have the same “cost” in terms of length and path latency. In ORNs, however, paths are *temporal* — equal length does not necessarily imply equal latency. While path lengths do impact the bandwidth “tax” and therefore overall network throughput [13], [14], [15], [12], [11], always preferring the shortest paths can leave a significant portion of network capacity idle [16]. Recent work, proposes a “uniform cost” approach that pins each flow to a single

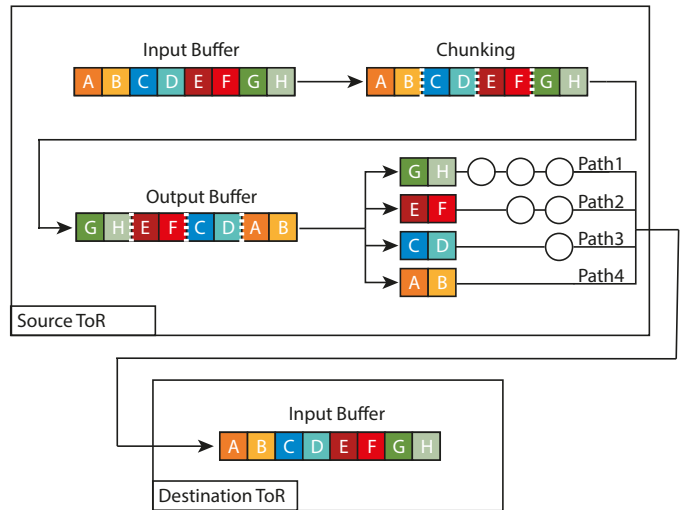


Fig. 1. Buffer chunking and distribution across output ports. Data from the input buffer is divided into fixed-size chunks, rearranged in the output buffer, and sent on the corresponding paths. Each colored-labeled block (A–H) represents a packet. Ultimately, the packets arrive in-order at the destination ToR.

path within each timeslot of the circuit switching schedule, mimicking ECMP semantics in static topologies [16]. However, similar to the inefficiencies of ECMP [17], [18], [19], this approach falls prey to under-utilization of the network, especially when the number of active flows is low and the flow size distribution is uniform.

Second, an alternative is to randomly spray packets across all available temporal paths, as in Valiant load balancing [2], [20]. While this can better utilize network capacity, it risks out-of-order packet delivery at the receiver — which is incompatible with existing datacenter architectures that predominantly rely on RoCEv2 and other transport protocols requiring strict in-order delivery [21], [22], [23], [24], [25], [26].

This paper is driven by a simple observation: multipath need not imply out-of-order *arrival* when transmission time and path latency can be controlled jointly. In an ORN, the periodic circuit schedule provides a nominal path-availability and latency structure. Queueing and processing variation can perturb this structure, so MPOA preserves ordering only for assignments that satisfy the bounded-error protection condition in Section III-C.

Figure 1 illustrates this perspective. For example, if a packet

sequence A, B, C, D arrives at the source ToR switch, the switch can transmit C and D earlier on a slower path and A and B later on a faster path. Provided that the scheduled arrival intervals exceed the relevant timing uncertainty, the receiver observes the original order A, B, C, D without downstream resequencing. ORNs provide a promising setting for this approach because their temporal paths expose diverse, schedule-derived latency characteristics.

We present MPOA, *Multiple Paths, Ordered Arrival*, a mechanism that reverses the transmission order of packet chunks at the source ToR, sending later chunks earlier on slower paths and earlier chunks later on faster paths. MPOA only reorders chunks within one circuit-schedule period, avoiding full-flow buffering while exploiting multiple temporal paths whenever the protection-interval condition can be met.

The idea got inspiration from the way people commonly park their cars in Japan. In accordance with some principles of Japanese culture, namely harmony (*wa*) and mutual consideration (*omoiyari*), people arriving early to work park further away from offices. This allows people that arrive late to find parking closer to their destination, which stops them from becoming even more late by having to walk longer.

Our preliminary simulations with 128 ToR switches show higher average link utilization than the recent UCMP baseline [16] in the evaluated, idealized setting. The simulation code and experimental artifacts are available at <https://github.com/inet-tub/TokyoRDCN>.

II. MOTIVATION

High link utilization while preserving ordered packet delivery is a key challenge in ORNs. In this section, we discuss the limitations of existing approaches and their implications, which motivate our work on MPOA.

Impact of skewed communication patterns: A perfectly uniform communication pattern across ToRs in oblivious reconfigurable networks can be easily routed using single-hop routing with high link utilization and full throughput. In contrast, prior work has shown that the worst-case throughput — i.e., the maximum sustainable load — of ORNs can be bounded by as low as 50% under skewed communication patterns, particularly ToR-to-ToR permutation traffic matrices [12], [11]. Such traffic patterns stress routing and load balancing algorithms, forcing the network to utilize all available paths to maximize throughput, which makes it significantly harder to preserve ordered delivery under these conditions.

Impact of flow size distribution: Recent work proposes UCMP for reconfigurable networks, designed in the spirit of ECMP for traditional networks. Unlike ECMP, UCMP relies on flow size for the choice of paths which makes flow size distribution a key factor in determining link utilization. In particular, many modern AI workloads feature a small set of flow sizes that repeat throughout the training process [27], [19], [28], [29], presenting unique challenges to rely on flow size for path selection and load balancing in ORNs.

Impact of reordering in existing datacenter fabrics: Many deployed RDMA networks use RoCEv2 NICs with strict in-

order or go-back- N receive semantics [26], [17], [21]; in such deployments, an out-of-order packet can trigger a NACK and retransmission of a packet sequence. Newer transports and NICs may provide greater reordering tolerance. MPOA targets deployments in which strict arrival order remains required or changing receiver and NIC behavior is undesirable.

Sender-side versus receiver-side ordering. A natural approach to preserve ordered delivery in a multipath network is to reorder packets at the destination ToR. However, this design has several drawbacks in reconfigurable datacenter fabrics, and MPOA deliberately shifts ordering responsibility to the sender ToR.

First, while receiver-side resequencing buffers can in principle be bounded, doing so requires the receiver to tolerate uncertainty in both path assignment and packet delay. In RDCNs, packets may arrive over multiple paths whose delays differ due to circuit scheduling and transient queueing. Without explicit knowledge of which path a packet traversed, the receiver must buffer packets until missing sequence numbers are either received or declared lost, which can inflate buffering requirements and delay loss recovery. In contrast, the sender ToR has precise knowledge of both path assignment and the circuit schedule, allowing it to arrange packet transmissions so that packets arrive in order under bounded delay variation.

Second, distinguishing between delayed and lost packets is inherently difficult at the receiver. A receiver observing out-of-order arrivals cannot easily infer whether a missing packet is delayed on a longer path or has been dropped, complicating both buffer management and transport-layer loss recovery even further. By scheduling packets with knowledge of expected arrival times, MPOA reduces this ambiguity and avoids reliance on large buffers or conservative timeouts.

Third, while MPOA introduces buffering at the sender, this buffering is explicitly bounded by the circuit scheduling period. MPOA only reorders packets within a scheduling epoch, ensuring that sender-side buffer requirements remain small and predictable. Receiver-side resequencing, by contrast, must accommodate the full range of delay variation across paths unless additional coordination mechanisms are introduced.

Finally, placing ordering logic at the sender ToR improves deployability and congestion signaling. Buffer pressure and scheduling decisions at the sender ToR can be communicated directly to end hosts and congestion control mechanisms, whereas buffer overflows at the receiver ToR are harder to signal promptly to senders. This makes sender-side ordering a natural fit for environments that prioritize predictable behavior, bounded buffering, and incremental deployment without requiring changes to NIC hardware or end-host transport stacks.

Receiver-side resequencing nevertheless remains a viable alternative when the destination ToR or NIC already provides reordering support and has sufficient path or schedule metadata. MPOA does not claim that source-side buffering is asymptotically smaller in every setting. Rather, it targets deployments that wish to preserve existing receiver behavior and can exploit path-assignment information already available at the source ToR. The choice is therefore an architectural

trade-off between source-side scheduling and destination-side buffering.

III. FLIPPING THE SCRIPT

Packets must be delivered to the end host in order, but they need not remain in that order while traversing the network. ORNs expose multiple temporal paths whose nominal delays can be predicted from the periodic circuit schedule. MPOA combines this schedule information with intentional source-side reordering: later chunks are transmitted earlier on slower paths, and earlier chunks are transmitted later on faster paths. If the selected paths and send times satisfy the protection-interval condition in Section III-C, the chunks arrive in order at the egress ToR.

Figure 1 illustrates the mechanism. The source ToR receives eight packets during one period and divides them into four in-order chunks. It assigns the final chunk, containing packets G and H, to the slowest path, and assigns progressively earlier chunks to progressively faster paths. The example preserves arrival order while using all four paths. The effect on flow completion time depends on the balance between path parallelism, source-side waiting, and protection intervals; we do not evaluate FCT in this work.

A. Chunking of Packets

To maximize link utilization, we aim to distribute packets across as many paths as possible. To achieve this, we need to determine which packets should be sent on which path. In the simplest case, the ingress ToR receives exactly as many packets for a given flow as there are paths to the destination. In this scenario, we can simply reverse the order of the packets: send the first packet on the slowest path, the second on the second-slowest path, and so on, until all packets have been dispatched. However, it is more likely that the number of packets received during a period exceeds the number of available paths. In such cases, instead of merely reversing the packet order, we must first chunk the buffered packets. The number of chunks should equal the number of available paths. Within each chunk, the packets remain in order. The chunks are then reversed and sent as described previously for individual packets. For now, we use static chunk sizes—for example, by leveraging known link capacities to calculate how many packets can be sent during each transmission period.

B. Calculating Path Order

MPOA requires a nominal latency estimate and an error bound for each selected temporal path, rather than perfect knowledge of its realized delay. In an ORN with a periodic schedule, a controller can derive the nominal path order directly from that schedule. For a path $p = (v_0, v_1, \dots, v_k)$ considered at time t , we model its nominal latency as

$$\ell(p, t) = \sum_{j=0}^{k-1} w_{(v_j, v_{j+1})}(t_j) + k d_{\text{proc}} + d_{\text{prop}}(p),$$

where $w_{(v_j, v_{j+1})}(t_j)$ is the wait until the required circuit becomes available after the packet reaches v_j . The value

t_j is updated hop by hop using the predicted arrival at each intermediate ToR. Thus, a one-hop path that becomes available late may be slower than a multi-hop path available immediately.

A centralized controller can precompute a ranked path set for every source–destination pair and schedule phase and install it at the source ToR. Alternatively, ToRs can maintain the ranking through lightweight probes. Dynamic queueing and model error are represented by e_i and its bound E_i in Section III-C; they need not trigger a new ranking for every packet. A controller can update rankings and error bounds at a slower timescale from observed prediction residuals. If residuals exceed the configured bound, the source can increase the protection interval, use fewer paths, or fall back to single-path forwarding.

C. Ordering Condition and Protection Intervals

MPOA requires the timing difference between consecutive chunks to absorb uncertainty in path delay. Consider two consecutive chunks C_i and C_{i+1} in the original packet order. Let s_i be the time at which the source starts transmitting C_i , let τ_i be its serialization time, and let ℓ_i be the nominal latency of its assigned path. The predicted arrival time of the last packet of C_i is therefore $s_i + \tau_i + \ell_i$.

Let e_i denote the *realized prediction error*: $e_i > 0$ means that C_i arrives later than predicted, while $e_i < 0$ means that it arrives earlier. The scheduler cannot know e_i in advance. Instead, it uses a non-negative bound E_i , associated with chunk C_i , its path, and the relevant schedule phase, such that $|e_i| \leq E_i$. Importantly, E_i is not an additional packet delay; it is a protection margin reserved by the scheduler. A conservative implementation can construct it from bounds on the individual sources of uncertainty:

$$E_i = E_{\text{sync}} + h_i E_{\text{proc}} + E_{\text{queue}, i} + E_{\text{model}, i},$$

where the terms respectively bound clock/schedule synchronization error, per-hop processing variation over h_i hops, transient queueing, and path-model error. Certified worst-case component bounds yield a deterministic guarantee. A bound derived from a measured latency percentile yields a corresponding statistical guarantee. If no useful finite bound is available, MPOA must use fewer paths or fall back to single-path forwarding.

For C_i to arrive before C_{i+1} under worst-case variation, it is sufficient that

$$s_i + \tau_i + \ell_i + E_i \leq s_{i+1} + \ell_{i+1} - E_{i+1}.$$

Equivalently, define the protection interval

$$g_i = (s_{i+1} + \ell_{i+1}) - (s_i + \tau_i + \ell_i).$$

Then the sufficient condition is

$$g_i \geq E_i + E_{i+1}.$$

The two bounds add because the worst case is that C_i arrives E_i late while C_{i+1} arrives E_{i+1} early.

Proposition 1. Assume that packets remain ordered within each chunk, $|e_i| \leq E_i$ for every chunk, and the inequality above holds for every adjacent pair. Then all chunks arrive in their original order.

Proof. The last packet of C_i arrives no later than $s_i + \tau_i + \ell_i + E_i$, while the first packet of C_{i+1} arrives no earlier than $s_{i+1} + \ell_{i+1} - E_{i+1}$. The condition therefore places the last packet of C_i no later than the first packet of C_{i+1} . Applying the same argument to every adjacent pair establishes the order of the complete chunk sequence. $\square$

Example. Suppose the predicted arrival time of the last packet of C_i is $10 \mu s$, while the predicted arrival time of the first packet of C_{i+1} is $16 \mu s$. The protection interval is therefore $g_i = 6 \mu s$. If $E_i = 2 \mu s$ and $E_{i+1} = 3 \mu s$, the worst-case arrival times are $12 \mu s$ and $13 \mu s$, respectively. Since $g_i = 6 \mu s \geq E_i + E_{i+1} = 5 \mu s$, the chunks remain ordered with $1 \mu s$ of residual slack. A path combination whose nominal arrivals were separated by only $4 \mu s$ would not be admitted under the same error bounds.

Larger protection intervals increase tolerance to delay uncertainty but reduce how much data can be packed into one schedule period.

Path-set admission. For each flow and scheduling epoch, MPOA first ranks the candidate temporal paths by nominal arrival time. It then selects the largest subset for which adjacent chunks can satisfy the protection-interval condition. If two candidate paths are too close in predicted arrival time, one of them is removed from the set. The buffered packets are divided among the remaining paths; when too few packets are available to populate all selected chunks, MPOA uses fewer paths rather than waiting solely to increase path diversity. If no multipath assignment satisfies the condition, the flow uses a single path during that epoch. Thus, MPOA applies multipathing opportunistically and falls back to conventional forwarding when timing uncertainty or insufficient buffered data makes ordered multipath transmission unsafe.

IV. RELATED WORK

While multipath protocols have been studied extensively in both academic and operational contexts, none of the existing ones are able to fully solve the gaps outlined in section II.

A. Reconfigurable and Optical Datacenter Networks

A large body of work has explored reconfigurable and circuit-switched datacenter fabrics, including optical circuit switching (OCS), rotor switches, and other time-varying topologies. These systems offer high bisection bandwidth and low power consumption but rely on fixed or semi-oblivious schedules that expose time-dependent path availability. Prior work has focused on topology design, schedule construction, and routing or load-balancing mechanisms that exploit these technologies, often using variants of equal-cost multipath routing (ECMP) such as the mentioned UCMP [8], [3], [16]. MPOA is largely orthogonal to this research.

B. Load Balancing and Packet Spraying in Datacenters

Packet spraying and Valiant-style load balancing are well-known techniques for improving utilization in datacenter topologies [20], [30]. However, spraying packets across multiple paths introduces packet reordering, which can degrade performance for transports that assume in-order delivery. To mitigate this, prior work has proposed flow pinning, flowlets, or receiver-side resequencing buffers [31], [32]. These approaches trade utilization for ordering guarantees or rely on buffering and reassembly at the receiver. MPOA explores a different point in this design space by arranging packets at the sender such that they arrive in order despite traversing multiple paths, reducing reliance on receiver-side reordering.

C. Multipath Transport Protocols and Scheduling

Multipath TCP (MPTCP) and related multipath transports have extensively studied how to schedule data across multiple subflows to improve throughput, robustness, and flow completion time [33], [34]. Existing literature proposes scheduler designs that account for RTT differences, congestion windows, and loss rates, often across paths with strongly varying characteristics, such as mobile and WiFi. This is done while typically assuming that the receiver can tolerate out-of-order delivery via sequence numbers and reassembly buffers. In contrast, MPOA addresses a different problem setting. First, it targets RDCNs where path latencies are largely determined by a known circuit schedule rather than having to be learned by end-to-end measurements. Second, its primary objective is not throughput optimization under tolerant receivers, but achieving in-order *arrival* under constrained receiver semantics. Finally, MPOA operates at the network edge (the sender ToR), rather than within an end-host transport stack. As such, MPOA complements rather than replaces prior work on multipath transport scheduling. Lastly, MPTCP and adjacent protocols often assume that multiple paths are visible on the end-host, e.g. in the form of multiple interfaces. However, from an end host's (i.e., a single unit in a server rack) perspective, only one interface (and thus path) is visible. The availability of multiple paths only starts at the ToR switch.

D. Receiver-Side Reordering and RDMA Transports

Modern datacenter transports, including RDMA-based protocols, vary widely in their tolerance for packet reordering. While newer NICs and transports increasingly support out-of-order arrivals, strict in-order delivery remains common in deployed systems due to simplicity, latency predictability, and loss recovery behavior [35], [36], [37]. Receiver-side reordering buffers have been proposed as a general solution to multipath-induced reordering, but they require buffering proportional to delay variation and complicate loss detection when path information is not explicitly available at the receiver [38]. MPOA shifts ordering responsibility to the sender ToR, where path assignment and schedule information are easily available, enabling bounded buffering and avoiding ambiguity between delayed and lost packets.

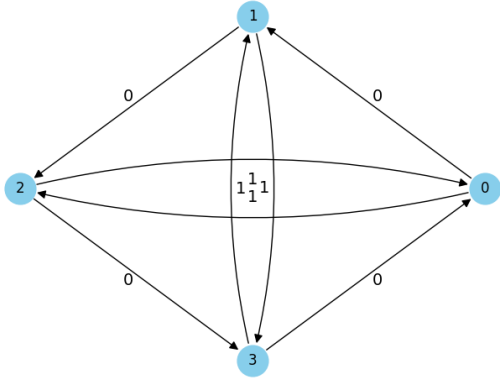


Fig. 2. Example topology with 4 ToRs and 2 reconfigurations per period.

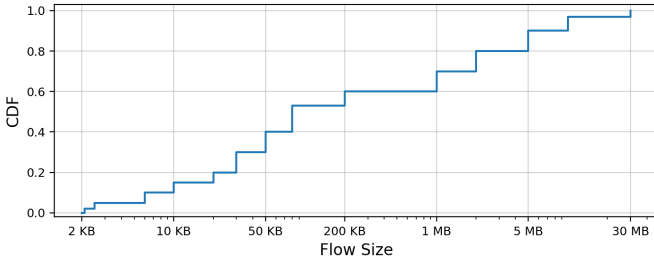


Fig. 3. CDF of flow sizes for the WebSearch workload.

V. EVALUATION

A. Methodology

To quantify the link-utilization potential of MPOA, we implemented an abstract, schedule-driven simulator in Python and compare it with the recent UCMP baseline. NetworkX generates ORN topologies of varying sizes. We evaluate batches of flows drawn from the WebSearch flow-size distribution in Figure 3, as well as constant-demand batches in which each flow offers 1 Gbps. Hosts are abstracted away: flows originate at ingress ToRs and terminate at randomly selected egress ToRs.

Each ToR generates the same number of flows, and all flows in a batch are released simultaneously. This synchronized release is intended to isolate spatial load-balancing behavior; it is not a model of production flow-arrival dynamics. We vary the number of reconfigurations per period, i.e., the number of different peers a ToR connects to before the schedule repeats. Figure 2 shows a small example. We enumerate temporal paths and their nominal schedule latencies. Figures 4 and 5 show the available path counts for 8 and 16 reconfigurations per period, respectively; Figures 6 and 7 show hop-count and nominal-latency diversity. All links have 10 Gbps capacity.

Evaluation scope. The simulator evaluates path allocation and average link utilization under an idealized, schedule-known model. It does not model stochastic queueing, packet loss, or link failures; equivalently, the reported runs use nominal delays and the zero-error case $e_i = E_i = 0$. The implementation used for the reported plots also does not exclude path combinations

using a positive protection interval. Because nonzero E_i values can remove some candidate path combinations, the reported MPOA utilization is an upper bound for a deployment that enforces nonzero protection intervals. The experiments do not measure flow completion time or empirically validate ordering under delay variation.

Baseline choice. UCMP is the primary baseline because, like MPOA, it aims to retain existing receiver behavior without destination-side resequencing. Packet spraying and Valiant load balancing occupy a different design point: they can use path diversity aggressively but do not preserve arrival order without additional receiver support. The UCMP comparison therefore evaluates the utilization cost of retaining this compatibility requirement; it is not a comparison against an unconstrained multipath upper bound.

We route each batch according to UCMP or the MPOA path-allocation abstraction in Algorithm 1 and calculate average link utilization. A complete scheduler would additionally admit only path/send-time combinations satisfying Section III-C. This filtering is not present in the reported utilization study, as stated in the evaluation scope above.

B. Results

Figures 8 and 9 report results for the constant-demand and WebSearch batches, respectively. MPOA achieves higher average link utilization than UCMP in these experiments, with the largest gap at low flow counts. Repeated flows with similar demands tend to select the same UCMP paths, leaving other

Algorithm 1 MPOA path-allocation abstraction used in the utilization study.

```

1: for each flow  $(s, d, \text{demand})$  do
2:   Find candidate temporal paths  $P$  from  $s$  to  $d$ 
3:   Sort  $P$  by descending latency
4:   Chunk packets of flow
5:   Reverse chunk order
6:    $\text{sent} \leftarrow 0$ 
7:   for  $t = 0$  to  $T_{\max}$  do
8:     if  $\text{sent} \geq \text{demand}$  then
9:       break
10:    end if
11:    for each path  $p \in P$  do
12:      Determine available capacity  $c$  along  $p$  at time
13:       $\text{send} \leftarrow \min(c, \text{demand} - \text{sent})$ 
14:      if  $\text{send} > 0$  then
15:        Update link usage along  $p$  at time  $t$ 
16:         $\text{sent} \leftarrow \text{sent} + \text{send}$ 
17:      end if
18:      if  $\text{sent} \geq \text{demand}$  then
19:        break
20:      end if
21:    end for
22:  end for
23: end for

```

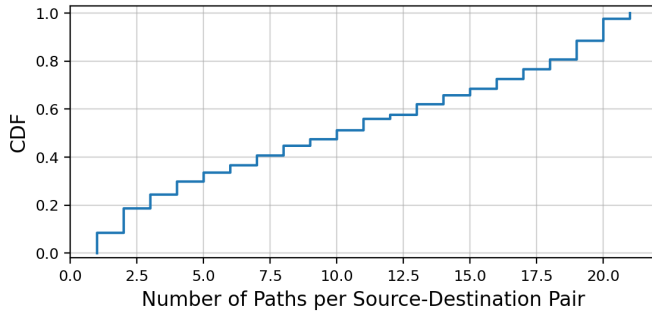


Fig. 4. CDF of number of paths for a topology with 128 ToRs and 8 reconfigurations per period.

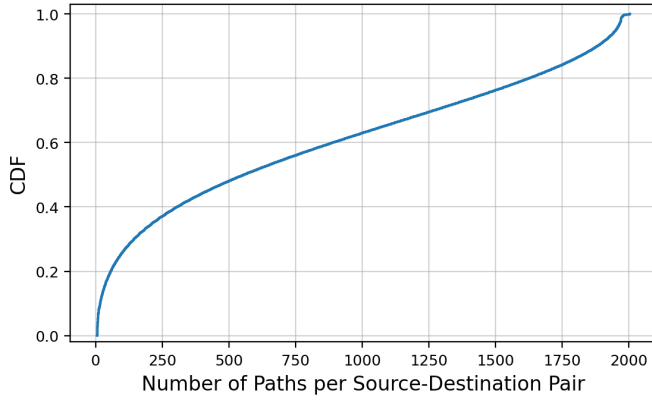


Fig. 5. CDF of number of paths for a topology with 128 ToRs and 16 reconfigurations per period.

temporal paths underused, whereas the MPOA abstraction distributes each flow over a larger candidate set. As the number and diversity of flows increase, the utilization gap narrows.

These results establish only the utilization potential of the path-allocation idea in the stated model. They do not demonstrate FCT improvements or robustness to stochastic delay, and they should be read together with the conditional ordering result in Section III-C. A priority mechanism that allocates more paths to selected traffic classes is a possible extension.

VI. OBSTACLES AND PITFALLS TO FULL IMPLEMENTATION

While our results demonstrate the potential benefits of intentionally modifying packet order within the network to leverage multiple paths, there are several interesting research questions that must be addressed before a full implementation or deployment can be realized.

One key challenge is the need for a mechanism that ensures intermediary ToRs are capable of forwarding all packets they receive from other ToRs. This is crucial, as any buffering of packets could lead to violations of the intended packet order at the egress ToR. Several potential approaches could address this issue. For example, traffic originating at a ToR could be separated from transit traffic, with the latter being allocated

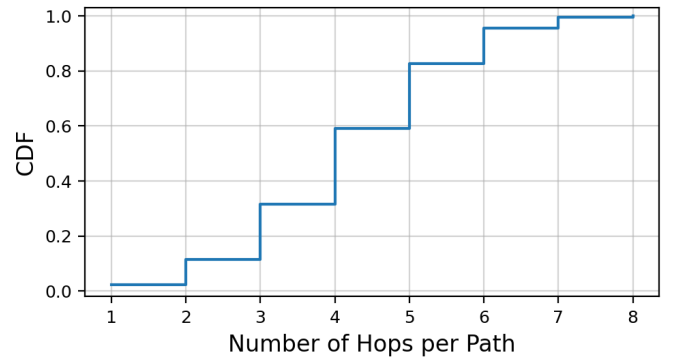


Fig. 6. CDF hop counts per path for a topology with 128 ToRs and 8 reconfigurations per period.

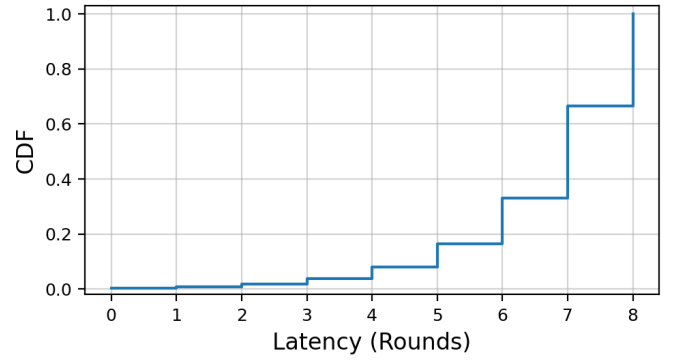


Fig. 7. CDF of latency of paths for a topology with 128 ToRs and 8 reconfigurations per period.

very shallow buffers to ensure that packets are dropped rather than delayed.

Transient queueing, synchronization error, and processing variation are represented by e_i and E_i in Section III-C. MPOA does not require a queue-free network, but a deterministic guarantee requires a defensible upper bound on these effects. If observed prediction residuals exceed the configured E_i , the source should enlarge the protection interval, reduce the number of simultaneous paths, or fall back to single-path forwarding. The current simulator does not evaluate this adaptation.

In particular paths should be selected in such a way that they are both useful for load balancing and sufficiently separated in predicted arrival time. This follows directly from the protection-interval condition in III-C. By choosing paths with larger temporal separation g_i can be increased, making the schedule more tolerant to possible delays. For example, rather than selecting the 10 fastest paths from a set of 100 available, it may be more useful to select paths at different points in the latency distribution, such as the 1st, 10th, 20th, and so on. This may sacrifice some raw path quality and slightly increase flow completion time, but it improves robustness by increasing the margin available to preserve in-order arrival.

Next, in a full implementation, it will be necessary to

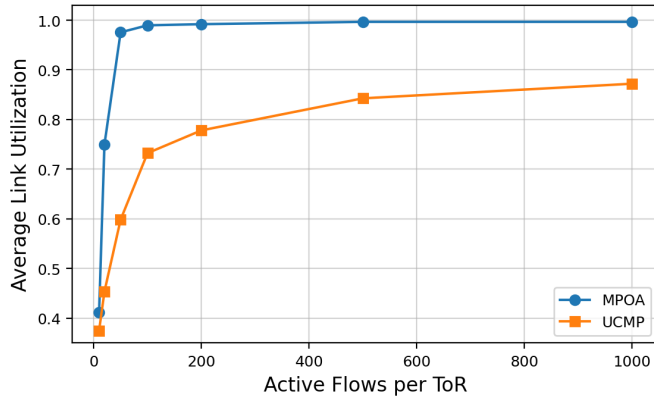


Fig. 8. Average link utilization on a topology with 128 ToRs and 8 reconfigurations per period for constant-demand flows offering 1 Gbps each.

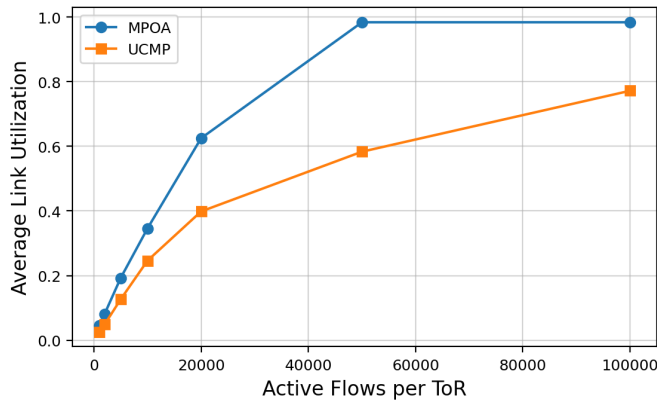


Fig. 9. Average link utilization on a topology with 128 ToRs and 8 reconfigurations per period for flows drawn from the WebSearch flow-size distribution.

carefully determine the chunk size c and the number of chunks n . Since we need to reverse the chunks before sending, all packets intended for transmission within a given period must already be present in the buffer. At a minimum, the following condition must be satisfied, where B_{period} denotes the number of packets a flow receives per period:

$$c \cdot n \leq B_{\text{period}}$$

If this condition is not met, the system would unnecessarily delay transmission, leading to increased latency. As the number of packets arriving in each period can vary, it would be reasonable to estimate B_{period} using smoothed averages, similar to how TCP estimates round-trip time (RTT). The exact algorithm for this estimation mechanism remains a subject for future work.

There is also an inherent trade-off between the chunk size and the number of chunks. The maximum chunk size is constrained by the link capacity of the path it is assigned to, while the number of chunks corresponds to the number of available paths. Although using more paths can improve link utilization by distributing the load more evenly, it also

increases the risk that a link failure may cause packet loss and, consequently, out-of-order delivery. Extensive experimentation will be needed to identify the optimal trade-offs and uncover potential hidden effects. This becomes particularly important if paths have differing capacities, which was not considered in our preliminary evaluation.

Another important concern is how to handle competing flows fairly. There are primarily two options. The first is to allocate a percentage of the available capacity to each flow in every period. The second is to allow only a small number of flows to utilize the full capacity during a given period, with the set of transmitting flows rotating across periods. The first approach ensures the fairest distribution of bandwidth but increases implementation complexity. In contrast, the second approach may be less fair—depending on the length of each period—but offers greater efficiency. Further studies are needed to determine whether the fairness provided by the second option is sufficient to justify its use.

Additionally, as discussed in Section III-B, we have so far assumed that a nominal path order can be computed from the circuit schedule. However, what constitutes the latency of a path remains an open question and requires further evaluation. In particular, reconfiguration delay plays a significant role. Depending on the underlying technology, this delay can range from tens of nanoseconds to several microseconds [16]. If the lower end of this range is achieved, processing delays at intermediate ToRs may become significant and should be taken into account when determining path order. This is an insight that can likely only be gained through real-world deployment. It can be incorporated into the system either by adding a fixed latency cost per hop or by shifting from latency-based metrics to a simpler hop-count-based approach.

Finally, as shown in Figure 5, the number of possible paths can grow rapidly with an increasing number of ToRs and reconfigurations per period. As such, computing and storing all available paths may become prohibitively expensive. Instead, it may be more practical to select a representative subset of paths to sample from the total. While this “path pruning” will likely reduce overall link utilization, it can significantly improve the feasibility of deployment. Future experiments will help characterize this trade-off. In any case, path selection should aim to maximize temporal separation between the chosen paths. For example, rather than selecting the 10 fastest paths from a set of 100, it may be more advantageous to select the 1st, 10th, 20th, and so on. This approach increases tolerance for timing variation and provides additional buffer to help ensure in-order delivery at the egress ToR.

VII. FUTURE WORK & CONCLUSION

In this work, we presented MPOA, a source-side scheduling mechanism for exploiting multiple temporal paths in ORNs. Sending later chunks earlier over slower paths and earlier chunks later over faster paths can improve average link utilization while preserving arrival order for assignments satisfying the bounded-error protection condition. Our simulations demonstrate the utilization potential of this idea relative to

UCMP in an idealized model with nominal path delays; they do not constitute a packet-level validation under queueing, loss, or failures.

Future work includes dynamic chunk sizing, adaptive error bounds and path selection, fairness and priority policies, and evaluation with realistic arrival processes, queueing, loss, and FCT. Relaxing strict ordering for traffic classes that tolerate a bounded amount of reordering may expose additional utilization opportunities.

ACKNOWLEDGMENT

The authors acknowledge the financial support by the Federal Ministry of Research, Technology and Space of Germany in the programme of “Souverän. Digital. Vernetzt.” Projects 6G-RIC, xG-RIC and xG-NOVA, project identification numbers 16KISK030, 16KIS2434, and 16KISS003 and in the programme “StartUpConnect” Project QUCast, project identification number 16KIS2650.

REFERENCES

- [1] M. Al-Fares, A. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” in *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*, ser. SIGCOMM '08. New York, NY, USA: Association for Computing Machinery, 2008, p. 63–74. [Online]. Available: <https://doi.org/10.1145/1402958.1402967>
- [2] H. Ballani, P. Costa, R. Behrendt, D. Cletheroe, I. Haller, K. Jozwik, F. Karinou, S. Lange, K. Shi, B. Thomsen, and H. Williams, “Sirius: A flat datacenter network with nanosecond optical switching,” in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 782–797. [Online]. Available: <https://doi.org/10.1145/3387514.3406221>
- [3] W. M. Mellette, R. McGuinness, A. Roy, A. Forench, G. Papen, A. C. Snoeren, and G. Porter, “Rotornet: A scalable, low-complexity, optical datacenter network,” in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 267–280. [Online]. Available: <https://doi.org/10.1145/3098822.3098838>
- [4] W. M. Mellette, A. Forench, R. Athapathu, A. C. Snoeren, G. Papen, and G. Porter, “Realizing rotornet: Toward practical microsecond scale optical networking,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 392–414. [Online]. Available: <https://doi.org/10.1145/3651890.3672273>
- [5] L. Poutievski, O. Mashayekhi, J. Ong, A. Singh, M. Tariq, R. Wang, J. Zhang, V. Beauregard, P. Conner, S. Gribble, R. Kapoor, S. Kratzer, N. Li, H. Liu, K. Nagaraj, J. Ornstein, S. Sawhney, R. Urata, L. Vicisano, K. Yasumura, S. Zhang, J. Zhou, and A. Vahdat, “Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking,” in *Proceedings of the ACM SIGCOMM 2022 Conference*, ser. SIGCOMM '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 66–85. [Online]. Available: <https://doi.org/10.1145/3544216.3544265>
- [6] M. Ghobadi, R. Mahajan, A. Phanishayee, N. Devanur, J. Kulkarni, G. Ranade, P.-A. Blanche, H. Rastegarfard, M. Glick, and D. Kilper, “Projector: Agile reconfigurable data center interconnect,” in *Proceedings of the 2016 ACM SIGCOMM Conference*, ser. SIGCOMM '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 216–229. [Online]. Available: <https://doi.org/10.1145/2934872.2934911>
- [7] K. Chen, A. Singla, A. Singh, K. Ramachandran, L. Xu, Y. Zhang, X. Wen, and Y. Chen, “Osa: An optical switching architecture for data center networks with unprecedented flexibility,” *IEEE/ACM Transactions on Networking*, vol. 22, no. 2, pp. 498–511, 2014.
- [8] N. Farrington, G. Porter, S. Radhakrishnan, H. H. Bazzaz, V. Subramanya, Y. Fainman, G. Papen, and A. Vahdat, “Helios: a hybrid electrical/optical switch architecture for modular data centers,” in *Proceedings of the ACM SIGCOMM 2010 Conference*, ser. SIGCOMM '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 339–350. [Online]. Available: <https://doi.org/10.1145/1851182.1851223>
- [9] C. Liang, X. Song, J. Cheng, M. Wang, Y. Liu, Z. Liu, S. Zhao, and Y. Cui, “Negotiator: Towards a simple yet effective on-demand reconfigurable datacenter network,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 415–432. [Online]. Available: <https://doi.org/10.1145/3651890.3672222>
- [10] D. Amir, N. Saran, T. Wilson, R. Kleinberg, V. Shrivastava, and H. Weatherspoon, “Shale: A practical, scalable oblivious reconfigurable network,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 449–464. [Online]. Available: <https://doi.org/10.1145/3651890.3672248>
- [11] D. Amir, T. Wilson, V. Shrivastava, H. Weatherspoon, R. Kleinberg, and R. Agarwal, “Optimal oblivious reconfigurable networks,” in *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, ser. STOC 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 1339–1352. [Online]. Available: <https://doi.org/10.1145/3519935.3520020>
- [12] V. Addanki, C. Avin, and S. Schmid, “Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 7, no. 1, mar 2023. [Online]. Available: <https://doi.org/10.1145/3579312>
- [13] A. Singla, P. B. Godfrey, and A. Kolla, “High throughput data center topology design,” in *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. Seattle, WA: USENIX Association, Apr. 2014, pp. 29–41. [Online]. Available: <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/singla>
- [14] P. Namyar, S. Supittayapornpong, M. Zhang, M. Yu, and R. Govindan, “A throughput-centric view of the performance of datacenter topologies,” in *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, ser. SIGCOMM '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 349–369. [Online]. Available: <https://doi.org/10.1145/3452296.3472913>
- [15] C. Griner, J. Zerwas, A. Blenk, M. Ghobadi, S. Schmid, and C. Avin, “Cerberus: The power of choices in datacenter topology design - a throughput perspective,” *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 5, no. 3, dec 2021. [Online]. Available: <https://doi.org/10.1145/3491050>
- [16] J. Li, H. Gong, F. De Marchi, A. Gong, Y. Lei, W. Bai, and Y. Xia, “Uniform-cost multi-path routing for reconfigurable data center networks,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 433–448.
- [17] A. Gangidi, R. Miao, S. Zheng, S. J. Bondu, G. Goes, H. Morsy, R. Puri, M. Riftadi, A. J. Shetty, J. Yang, S. Zhang, M. J. Fernandez, S. Gandham, and H. Zeng, “Rdma over ethernet for distributed training at meta scale,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 57–70. [Online]. Available: <https://doi.org/10.1145/3651890.3672233>
- [18] K. Qian, Y. Xi, J. Cao, J. Gao, Y. Xu, Y. Guan, B. Fu, X. Shi, F. Zhu, R. Miao, C. Wang, P. Wang, P. Zhang, W. Bai, and Y. Xia, “Alibaba hpn: A data center network for large language model training,” in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 691–706. [Online]. Available: <https://doi.org/10.1145/3651890.3672265>
- [19] V. Addanki, P. Goyal, I. Marinos, and S. Schmid, “Ethereal: Divide and conquer network load balancing in large-scale distributed training,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.00550>
- [20] L. G. Valiant, “A scheme for fast parallel communication,” *SIAM journal on computing*, vol. 11, no. 2, pp. 350–361, 1982.
- [21] W. Bai, S. S. Abdeen, A. Agrawal, K. K. Attre, P. Bahl, A. Bhagat, G. Bhaskara, T. Brokhman, L. Cao, A. Cheema, R. Chow, J. Cohen, M. Elhaddad, V. Ette, I. Figlin, D. Firestone, M. George, I. German, L. Ghai, E. Green, A. Greenberg, M. Gupta, R. Haagens, M. Hendel, R. Howlader, N. John, J. Johnstone, T. Jolly, G. Kramer, D. Kruse, A. Kumar, E. Lan, I. Lee, A. Levy, M. Lipshteyn, X. Liu, C. Liu, G. Lu, Y. Lu, X. Lu, V. Makhervaks, U. Malashanka, D. A.

- Maltz, I. Marinos, R. Mehta, S. Murthi, A. Namdhari, A. Ogus, J. Padhye, M. Pandya, D. Phillips, A. Power, S. Puri, S. Raindel, J. Rhee, A. Russo, M. Sah, A. Sheriff, C. Sparacino, A. Srivastava, W. Sun, N. Swanson, F. Tian, L. Tomczyk, V. Vadlamuri, A. Wolman, Y. Xie, J. Yom, L. Yuan, Y. Zhang, and B. Zill, "Empowering azure storage with RDMA," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 49–67. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/bai>
- [22] G. Kumar, N. Dukkupati, K. Jang, H. M. G. Wassel, X. Wu, B. Montazeri, Y. Wang, K. Springborn, C. Alfeld, M. Ryan, D. Wetherall, and A. Vahdat, "Swift: Delay is simple and effective for congestion control in the datacenter," in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 514–528. [Online]. Available: <https://doi.org/10.1145/3387514.3406591>
- [23] Y. Li, R. Miao, H. H. Liu, Y. Zhuang, F. Feng, L. Tang, Z. Cao, M. Zhang, F. Kelly, M. Alizadeh, and M. Yu, "Hppc: High precision congestion control," in *Proceedings of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 44–58. [Online]. Available: <https://doi.org/10.1145/3341302.3342085>
- [24] R. Mittal, V. T. Lam, N. Dukkupati, E. Blem, H. Wassel, M. Ghobadi, A. Vahdat, Y. Wang, D. Wetherall, and D. Zats, "Timely: Rtt-based congestion control for the datacenter," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, ser. SIGCOMM '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 537–550. [Online]. Available: <https://doi.org/10.1145/2785956.2787510>
- [25] V. Addanki, O. Michel, and S. Schmid, "PowerTCP: Pushing the performance limits of datacenter networks," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. Renton, WA: USENIX Association, Apr. 2022, pp. 51–70. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/addanki>
- [26] Y. Zhu, H. Eran, D. Firestone, C. Guo, M. Lipshteyn, Y. Liron, J. Padhye, S. Raindel, M. H. Yahia, and M. Zhang, "Congestion control for large-scale rdma deployments," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, ser. SIGCOMM '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 523–536. [Online]. Available: <https://doi.org/10.1145/2785956.2787484>
- [27] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, "TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 739–767. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wang-weiyang>
- [28] W. Wang, M. Ghobadi, K. Shakeri, Y. Zhang, and N. Hasani, "Rail-only: A low-cost high-performance network for training llms with trillion parameters," in *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*, 2024, pp. 1–10.
- [29] W. Li, X. Liu, Y. Li, Y. Jin, H. Tian, Z. Zhong, G. Liu, Y. Zhang, and K. Chen, "Understanding communication characteristics of distributed training," in *Proceedings of the 8th Asia-Pacific Workshop on Networking*, ser. APNet '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1–8. [Online]. Available: <https://doi.org/10.1145/3663408.3663409>
- [30] K. He, E. Rozner, K. Agarwal, Y. Guo, B. Felderman, J. Carter, and A. Akella, "Presto: Edge-based load balancing for fast datacenter networks," in *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM '15)*, 2015.
- [31] S. Sinha, S. Kandula, and D. Katabi, "Harnessing tcp's burstiness with flowlet switching," in *Proceedings of the 3rd ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-III)*, 2004.
- [32] A. Dixit, P. Prakash, Y. C. Hu, and R. R. Kompella, "On the impact of packet spraying in data center networks," in *Proceedings of IEEE INFOCOM*, 2013, pp. 2130–2138.
- [33] O. Bonaventure, M. Handley, C. Raiciu, A. Ford, and C. Paasch, "RFC 8684: Tcp extensions for multipath operation with multiple addresses," RFC Editor, Mar. 2020, rFC 8684.
- [34] O. Bonaventure, Q. De Coninck, C. Paasch, M. Amend, M. Baerts, and M. Piroux, "Multipath schedulers," Internet-Draft (IETF), draft-bonaventure-iccrs-schedulers-02, Oct. 2021, expired Internet-Draft.
- [35] InfiniBand Trade Association (IBTA), "InfiniBand Architecture supplement: RoCEv2 (IP Routable RoCE)," InfiniBand Trade Association, Tech. Rep., Sep. 2014, specification release; commonly referenced as Annex A17 / RoCEv2. [Online]. Available: <https://www.infinibandta.org/ibta-announces-new-roce-specification/>
- [36] D. A. Reed, R. Elzur, R. M. Alverson, and T. Talpey, *RDMA Aware Networks Programming User Manual*. Prentice Hall, 2005.
- [37] R. Mittal, A. Shpiner, A. Panda, E. Zahavi, A. Krishnamurthy, S. Ratnasamy, and S. Shenker, "Revisiting network support for RDMA," arXiv preprint, 2018. [Online]. Available: <https://arxiv.org/abs/1806.08159>
- [38] Y. Geng, V. Jeyakumar, A. Kabbani, and M. Alizadeh, "Juggler: A practical reordering-resilient network stack for datacenters," in *Proceedings of the 11th European Conference on Computer Systems (EuroSys '16)*, 2016.